

Assignment - EDA using Pandas

Instructions

- Clone a copy of the assignment notebook in Google Colab.
- **Complete the tasks by filling in the code where indicated by TODO comments. Put each task in a new cell.**
- Run each cell to ensure your code works correctly.
- Pay attention to the output of each cell and interpret the results.
- For visualisation tasks, make sure to label axes, add titles, and choose appropriate colour schemes.
- After completing all tasks, review your findings and include notes of your insights gained from the EDA process as markdown notes for assessments.

```
# Titanic Dataset Exploratory Data Analysis

# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# =====
# Task 1: Load the Titanic dataset
# =====
print("="*60)
print("TASK 1: Load the Titanic dataset")
print("="*60)

# Load the Titanic dataset from the URL
url = "https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv"
df = pd.read_csv(url)

print(f"Dataset loaded successfully with {df.shape[0]} rows and {df.shape[1]} columns")
print(f"Columns: {df.columns.tolist()}")

# =====
# Task 2: Inspect the data
# =====
print("\n" + "="*60)
print("TASK 2: Inspect the data")
print("="*60)

# Display the first 5 rows
print("First 5 rows:")
print(df.head())

# Display the last 5 rows
print("\nLast 5 rows:")
```

```

print("\nLast 5 rows:")
print(df.tail())

# Display data types
print("\nData types of each column:")
print(df.dtypes)

# Display summary statistics
print("\nSummary statistics of numerical columns:")
print(df.describe())

# =====
# Task 3: Clean and preprocess the data
# =====
print("\n" + "="*60)
print("TASK 3: Clean and preprocess the data")
print("="*60)

# Create a new categorical feature 'PclassCategory' from 'Pclass'
class_mapping = {1: 'First', 2: 'Second', 3: 'Third'}
df['PclassCategory'] = df['Pclass'].map(class_mapping)
print("Created PclassCategory feature")

# The 'Cabin' column is not present in the dataset, so we skip creating 'CabinCategory'
# If it were present, the original code would be:
# def get_cabin_category(cabin):
#     if pd.isna(cabin):
#         return 'Unknown'
#     elif cabin[0].isdigit():
#         return 'Unknown'
#     else:
#         return cabin[0]
# df['CabinCategory'] = df['Cabin'].apply(get_cabin_category)
# print("Created CabinCategory feature")

# Handle missing values in the 'Age' column (use mean age)
mean_age = df['Age'].mean()
df['Age'] = df['Age'].fillna(mean_age)
print(f"Filled missing Age values with mean age: {mean_age:.2f}")

# Create a 'Family' feature by combining 'SibSp' and 'Parch'
df['Family'] = df['Siblings/Spouses Aboard'] + df['Parents/Children Aboard']
print("Created Family feature (SibSp + Parch)")

print("\nData after preprocessing:")
# Removed 'CabinCategory' from the print statement as it's not created
print(df[['Name', 'Age', 'PclassCategory', 'Family']].head())

# =====
# Task 4: Identify specific data points
# =====
print("\n" + "="*60)
print("TASK 4: Identify specific data points")
print("="*60)

```

```

# Find passengers with the highest fare
highest_fare = df['Fare'].max()
highest_fare_passengers = df[df['Fare'] == highest_fare]
print(f"Highest fare: ${highest_fare:.2f}")
print("Passengers with the highest fare:")
print(highest_fare_passengers[['Name', 'Fare', 'Pclass']])

# Find passengers with the largest family size
largest_family = df['Family'].max()
largest_family_passengers = df[df['Family'] == largest_family]
print(f"\nLargest family size: {largest_family}")
print("Passengers with the largest family size:")
print(largest_family_passengers[['Name', 'Family', 'Siblings/Spouses Aboard', 'Parent/Children Aboard']])

# =====
# Task 5: Demonstrate Pandas Series and DataFrame functionalities
# =====
print("\n" + "="*60)
print("TASK 5: Demonstrate Pandas Series and DataFrame functionalities")
print("="*60)

# Create a Pandas Series of passenger names
names_series = df['Name']
print(f"Names Series (first 5):\n{names_series.head()}")
print(f"Type: {type(names_series)}")

# Create a DataFrame of passenger names and ages
names_ages_df = df[['Name', 'Age']]
print(f"\nNames and Ages DataFrame (first 5):\n{names_ages_df.head()}")
print(f"Shape: {names_ages_df.shape}")

# Slice the DataFrame to show only passengers older than 50
older_passengers = df[df['Age'] > 50]
print(f"\nPassengers older than 50 (count: {len(older_passengers)}):")
print(older_passengers[['Name', 'Age', 'Pclass']].head())

# =====
# Task 6: Introduce multi-indexing
# =====
print("\n" + "="*60)
print("TASK 6: Introduce multi-indexing")
print("="*60)

# Create a multi-index DataFrame grouped by 'Sex' and 'Pclass'
multi_index_df = df.groupby(['Sex', 'Pclass']).agg({
    'Survived': ['mean', 'count'],
    'Age': 'mean',
    'Fare': 'mean'
})
print("Multi-index DataFrame (Survival rates by Sex and Pclass):")
print(multi_index_df)

# =====

```

```

# Task 7: Visualizations
# =====
print("\n" + "="*60)
print("TASK 7: Creating Visualizations")
print("="*60)

# Set style for better looking plots
sns.set_style('whitegrid')

# 7.1: Histogram of passenger ages
plt.figure(figsize=(10, 6))
plt.hist(df['Age'], bins=20, color='skyblue', edgecolor='black', alpha=0.7)
plt.title('Histogram of Passenger Ages', fontsize=14, fontweight='bold')
plt.xlabel('Age')
plt.ylabel('Count')
plt.axvline(df['Age'].mean(), color='red', linestyle='dashed', linewidth=2, label=f'Mean')
plt.axvline(df['Age'].median(), color='green', linestyle='dashed', linewidth=2, label=f'Median')
plt.legend()
plt.tight_layout()
plt.show()

# 7.2: Bar plot of survival rates by passenger class
survival_by_class = df.groupby('Pclass')['Survived'].mean()
plt.figure(figsize=(8, 6))
survival_by_class.plot(kind='bar', color=['#FF6B6B', '#4ECDC4', '#45B7D1'])
plt.title('Survival Rates by Passenger Class', fontsize=14, fontweight='bold')
plt.xlabel('Passenger Class')
plt.ylabel('Survival Rate')
plt.xticks(rotation=0)
plt.ylim(0, 1)
for i, v in enumerate(survival_by_class):
    plt.text(i, v + 0.02, f'{v:.2%}', ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

# 7.3: Heatmap of correlation between numerical features
numeric_cols = ['Survived', 'Pclass', 'Age', 'Siblings/Spouses Aboard', 'Parents/Children Aboard']
correlation_matrix = df[numeric_cols].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.3f', linewidths=0)
plt.title('Correlation Heatmap of Numerical Features', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 7.4: Pair plot of selected features
selected_features = ['Age', 'Fare', 'Siblings/Spouses Aboard', 'Parents/Children Aboard']
sns.pairplot(df, vars=selected_features, hue='Survived', palette=['red', 'green'], dropdiagonal=True)
plt.suptitle('Pair Plot of Selected Features by Survival', y=1.02, fontsize=14, fontweight='bold')
plt.show()

# =====
# Task 8: Analyze survival rates
# =====
print("\n" + "="*60)

```

```

print("TASK 8: Analyze survival rates")
print("="*60)

# Survival rates by sex
survival_by_sex = df.groupby('Sex')['Survived'].mean()
print("Survival rates by sex:")
print(survival_by_sex)

# Survival rates by passenger class
survival_by_class = df.groupby('Pclass')['Survived'].mean()
print("\nSurvival rates by passenger class:")
print(survival_by_class)

# Additional survival analysis
survival_sex_class = df.groupby(['Sex', 'Pclass'])['Survived'].mean()
print("\nSurvival rates by sex and passenger class:")
print(survival_sex_class)

# =====
# Task 9: Analyze age distribution
# =====
print("\n" + "="*60)
print("TASK 9: Analyze age distribution")
print("="*60)

# Box plot of age distribution by passenger class and sex
plt.figure(figsize=(12, 8))
sns.boxplot(x='Pclass', y='Age', hue='Sex', data=df, palette=['skyblue', 'pink'])
plt.title('Age Distribution by Passenger Class and Sex', fontsize=14, fontweight='bold')
plt.xlabel('Passenger Class')
plt.ylabel('Age')
plt.legend(title='Sex')
plt.tight_layout()
plt.show()

# Age statistics by class and sex
age_stats = df.groupby(['Pclass', 'Sex'])['Age'].describe()
print("Age statistics by passenger class and sex:")
print(age_stats)

# =====
# Task 10: Analyze fare distribution
# =====
print("\n" + "="*60)
print("TASK 10: Analyze fare distribution")
print("="*60)

# Violin plot of fare distribution by passenger class and survival status
plt.figure(figsize=(12, 8))
sns.violinplot(x='Pclass', y='Fare', hue='Survived', data=df, split=True,
               palette={0: '#FF6B6B', 1: '#4ECDC4'})
plt.title('Fare Distribution by Passenger Class and Survival Status', fontsize=14, fontweight='bold')
plt.xlabel('Passenger Class')
plt.ylabel('Fare ($)')

```

```

plt.legend(title='Survived', labels=['No', 'Yes'])
plt.tight_layout()
plt.show()

# Fare statistics by class and survival
fare_stats = df.groupby(['Pclass', 'Survived'])['Fare'].describe()
print("Fare statistics by passenger class and survival status:")
print(fare_stats)

# =====
# Summary
# =====
print("\n" + "="*60)
print("EDA SUMMARY")
print("="*60)
print(f"Total passengers: {len(df)}")
print(f"Overall survival rate: {df['Survived'].mean():.2%}")
print(f"Average age: {df['Age'].mean():.1f} years")
print(f"Average fare: ${df['Fare'].mean():.2f}")
print(f"Female survival rate: {survival_by_sex['female']:.2%}")
print(f"Male survival rate: {survival_by_sex['male']:.2%}")
print(f"First class survival rate: {survival_by_class[1]:.2%}")
print(f"Third class survival rate: {survival_by_class[3]:.2%}")
print("\nEDA tasks completed successfully!")

# Show final DataFrame structure
print("\nFinal DataFrame shape:", df.shape)
print("Columns:", df.columns.tolist())

```

```
=====
TASK 1: Load the Titanic dataset
=====
Dataset loaded successfully with 887 rows and 8 columns
Columns: ['Survived', 'Pclass', 'Name', 'Sex', 'Age', 'Siblings/Spouses Aboard', 'Pa

=====
TASK 2: Inspect the data
=====
First 5 rows:
      Survived  Pclass                                Name \
0           0        3                                Mr. Owen Harris Braund
1           1        1  Mrs. John Bradley (Florence Briggs Thayer) Cum...
2           1        3                                Miss. Laina Heikkinen
3           1        1      Mrs. Jacques Heath (Lily May Peel) Futrelle
4           0        3                                Mr. William Henry Allen

      Sex  Age  Siblings/Spouses Aboard  Parents/Children Aboard    Fare
0  male  22.0                1                0    7.2500
1  female  38.0                1                0   71.2833
2  female  26.0                0                0    7.9250
3  female  35.0                1                0   53.1000
4  male  35.0                0                0    8.0500

Last 5 rows:
      Survived  Pclass                                Name    Sex  Age \
882           0        2      Rev. Juozas Montvila    male  27.0
883           1        1  Miss. Margaret Edith Graham  female  19.0
884           0        3  Miss. Catherine Helen Johnston  female   7.0
885           1        1      Mr. Karl Howell Behr    male  26.0
886           0        3      Mr. Patrick Dooley    male  32.0

      Siblings/Spouses Aboard  Parents/Children Aboard    Fare
882                0                0   13.00
883                0                0   30.00
884                1                2   23.45
885                0                0   30.00
886                0                0    7.75

Data types of each column:
Survived          int64
Pclass            int64
Name              object
Sex               object
Age              float64
Siblings/Spouses Aboard  int64
Parents/Children Aboard  int64
Fare              float64
dtype: object

Summary statistics of numerical columns:
      Survived  Pclass  Age  Siblings/Spouses Aboard \
count  887.000000  887.000000  887.000000    887.000000
mean    0.385569    2.305524   29.471443     0.525366
std     0.487004    0.836662   14.121908     1.104669
min     0.000000    1.000000    0.420000     0.000000
25%     0.000000    2.000000   20.250000     0.000000
50%     0.000000    3.000000   28.000000     0.000000
75%     1.000000    3.000000   38.000000     1.000000
```

15%	1.0000000	3.0000000	38.0000000	1.0000000
max	1.0000000	3.0000000	80.0000000	8.0000000

	Parents/Children Aboard	Fare
count	887.000000	887.000000
mean	0.383315	32.30542
std	0.807466	49.78204
min	0.000000	0.000000
25%	0.000000	7.92500
50%	0.000000	14.45420
75%	0.000000	31.13750
max	6.000000	512.32920

```
=====
TASK 3: Clean and preprocess the data
=====
```

```
Created PclassCategory feature
Filled missing Age values with mean age: 29.47
Created Family feature (SibSp + Parch)
```

Data after preprocessing:

	Name	Age	PclassCategory
0	Mr. Owen Harris Braund	22.0	Third
1	Mrs. John Bradley (Florence Briggs Thayer) Cum...	38.0	First
2	Miss. Laina Heikkinen	26.0	Third
3	Mrs. Jacques Heath (Lily May Peel) Futrelle	35.0	First
4	Mr. William Henry Allen	35.0	Third

	Family
0	1
1	1
2	0
3	1
4	0

TASK 4: Identify specific data points

Highest fare: \$512.33

Passengers with the highest fare:

	Name	Fare	Pclass
257	Miss. Anna Ward	512.3292	1
676	Mr. Thomas Drake Martinez Cardeza	512.3292	1
733	Mr. Gustave J Lesurer	512.3292	1

Largest family size: 10

Passengers with the largest family size:

	Name	Family	Siblings/Spouses	Aboard
158	Master. Thomas Henry Sage	10		8
179	Miss. Constance Gladys Sage	10		8
200	Mr. Frederick Sage	10		8
322	Mr. George John Jr Sage	10		8
788	Miss. Stella Anna Sage	10		8
842	Mr. Douglas Bullen Sage	10		8
859	Miss. Dorothy Edith Sage	10		8

158	Parents/Children Aboard	2
179		2

```

200
322
788
842
859

```

TASK 5: Demonstrate Pandas Series and DataFrame functionalities

Names Series (first 5):

```

0      Mr. Owen Harris Braund
1  Mrs. John Bradley (Florence Briggs Thayer) Cum...
2      Miss. Laina Heikkinen
3  Mrs. Jacques Heath (Lily May Peel) Futrelle
4      Mr. William Henry Allen

```

Name: Name, dtype: object

Type: <class 'pandas.core.series.Series'>

Names and Ages DataFrame (first 5):

		Name	Age
0		Mr. Owen Harris Braund	22.0
1	Mrs. John Bradley (Florence Briggs Thayer) Cum...		38.0
2		Miss. Laina Heikkinen	26.0
3	Mrs. Jacques Heath (Lily May Peel) Futrelle		35.0
4		Mr. William Henry Allen	35.0

Shape: (887, 2)

Passengers older than 50 (count: 72):

	Name	Age	Pclass
6	Mr. Timothy J McCarthy	54.0	1
11	Miss. Elizabeth Bonnell	58.0	1
15	Mrs. (Mary D Kingcome) Hewlett	55.0	2
33	Mr. Edward H Wheadon	66.0	2
53	Mr. Engelhart Cornelius Ostby	65.0	1

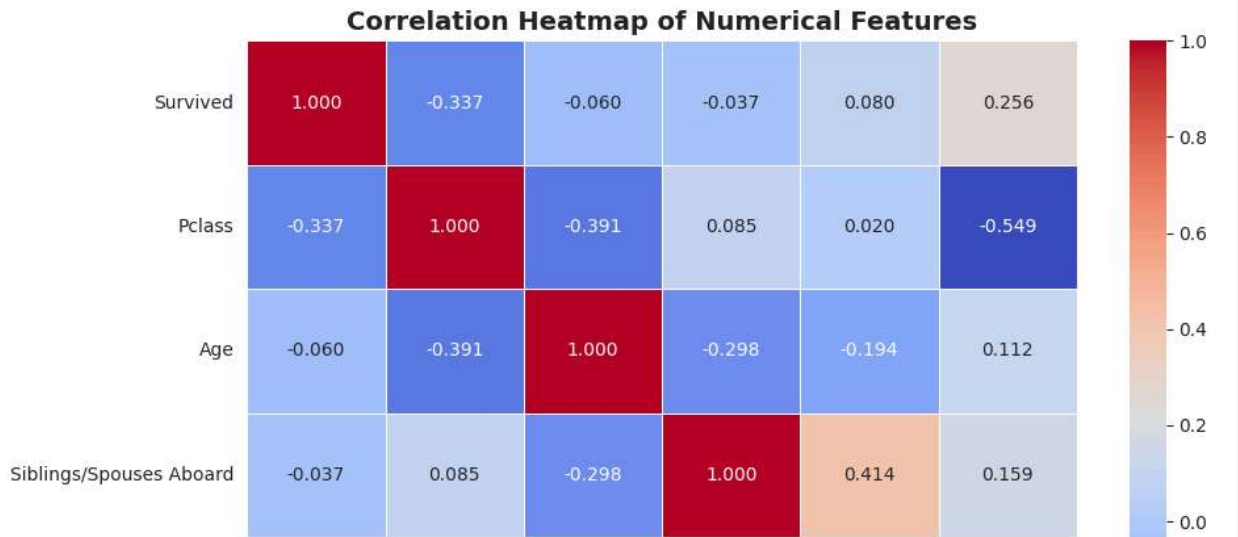
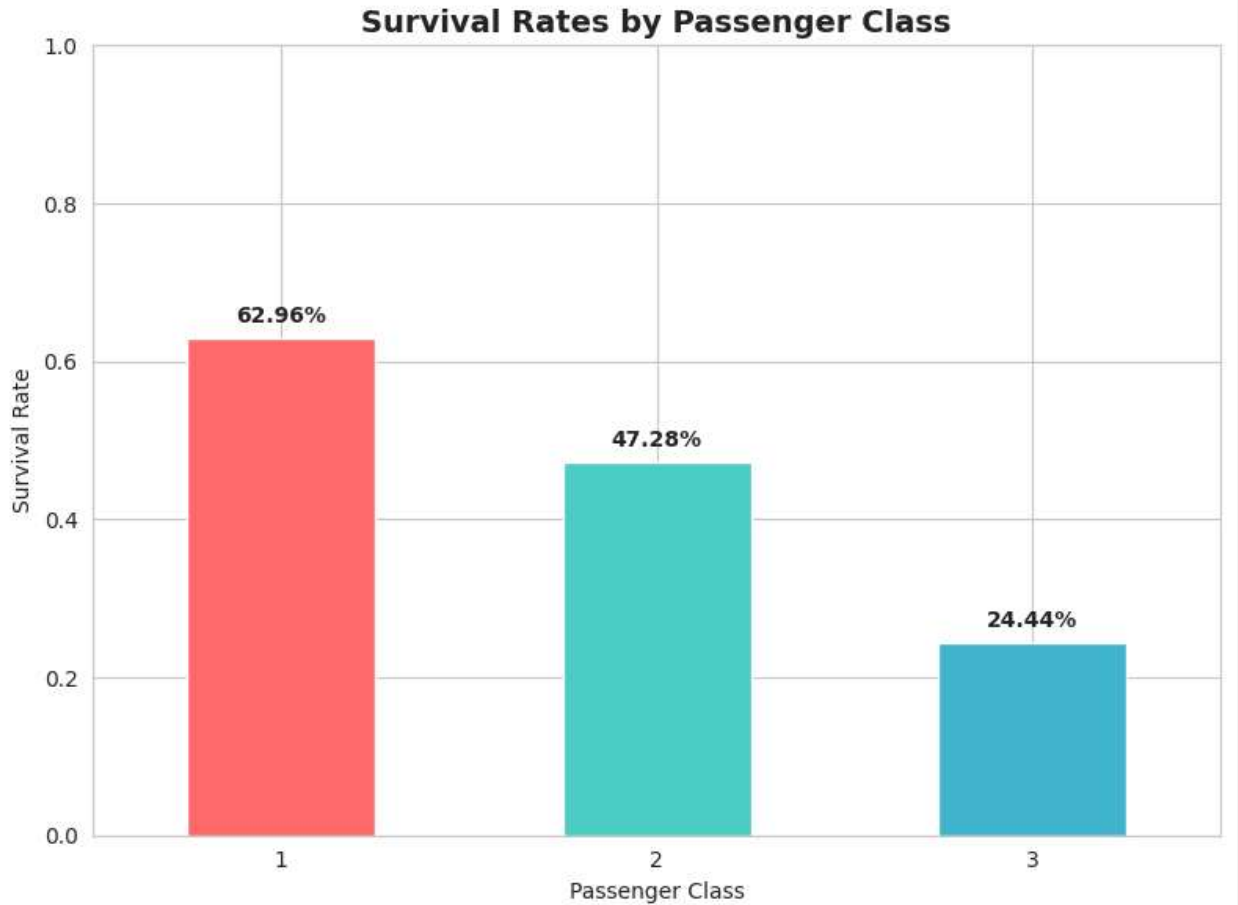
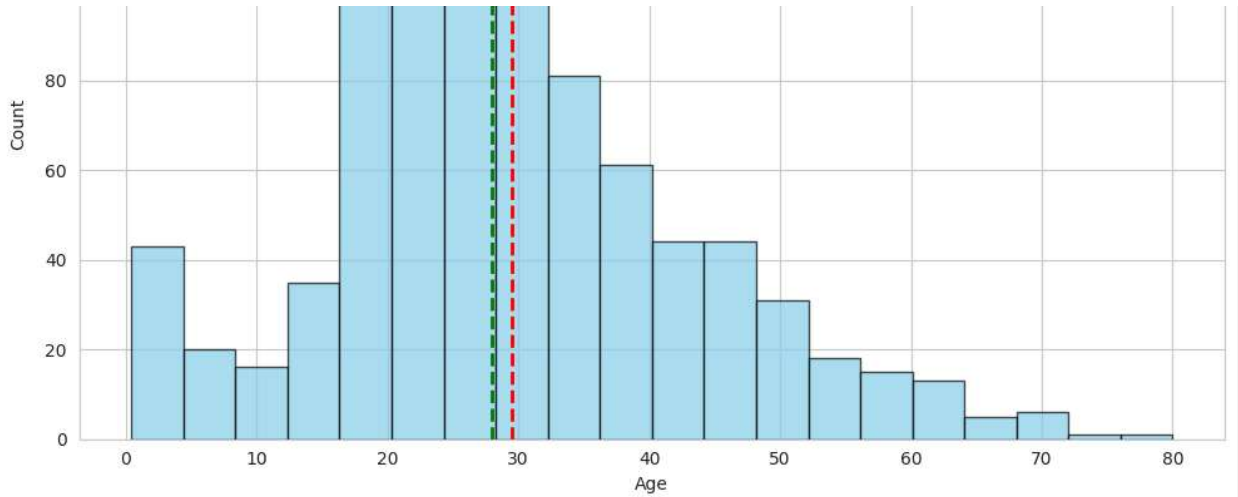
TASK 6: Introduce multi-indexing

Multi-index DataFrame (Survival rates by Sex and Pclass):

		Survived		Age	Fare
		mean	count	mean	mean
Sex	Pclass				
female	1	0.968085	94	35.255319	106.125798
	2	0.921053	76	28.980263	21.970121
	3	0.500000	144	22.135417	16.118810
male	1	0.368852	122	41.511639	67.226127
	2	0.157407	108	30.493796	19.741782
	3	0.137026	343	26.470612	12.695466

TASK 7: Creating Visualizations







```
Name: Survived, dtype: float64
```

Survival rates by passenger class:

```
Pclass
```

```
1    0.629630
```

```
2    0.472826
```

```
3    0.244353
```

```
Name: Survived, dtype: float64
```

Survival rates by sex and passenger class:

```
Sex    Pclass
```

```
female 1    0.968085
```

```
        2    0.921053
```

```
        3    0.500000
```

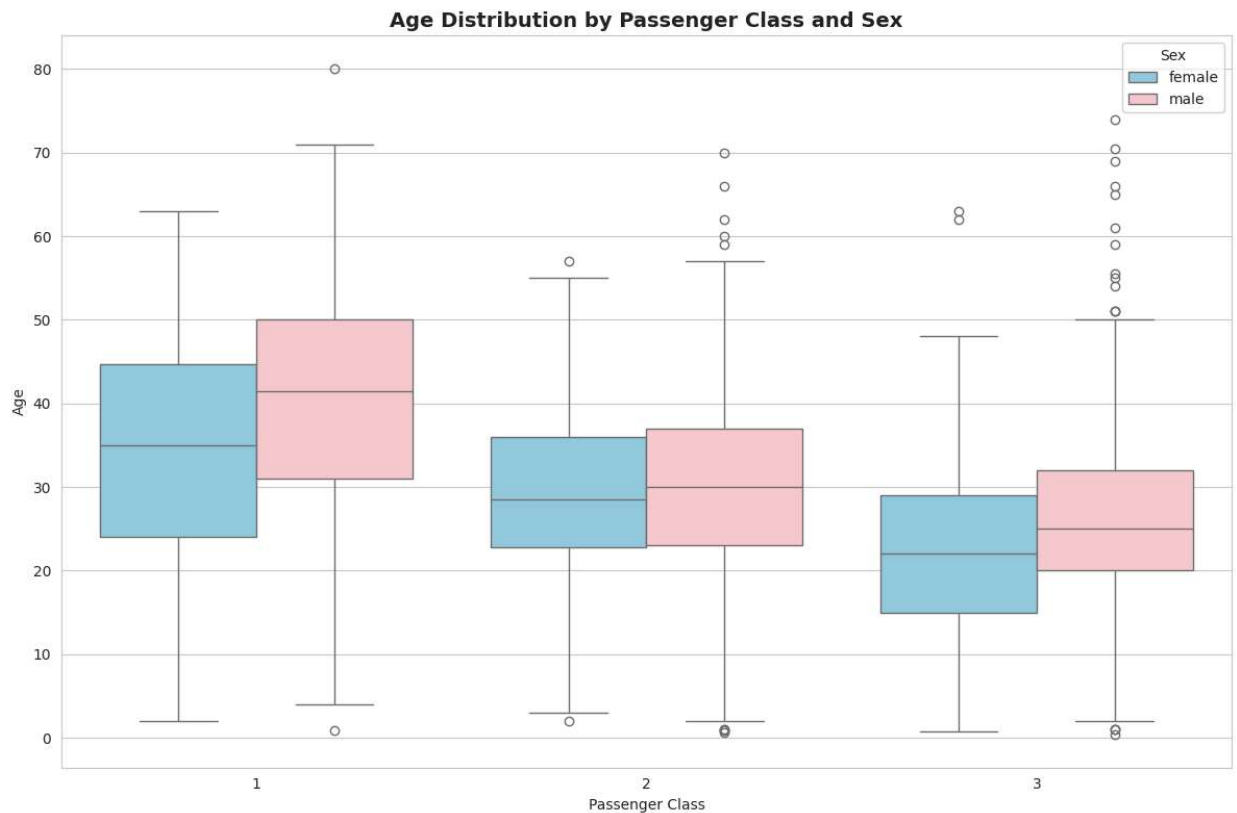
```
male   1    0.368852
```

```
        2    0.157407
```

```
        3    0.137026
```

```
Name: Survived, dtype: float64
```

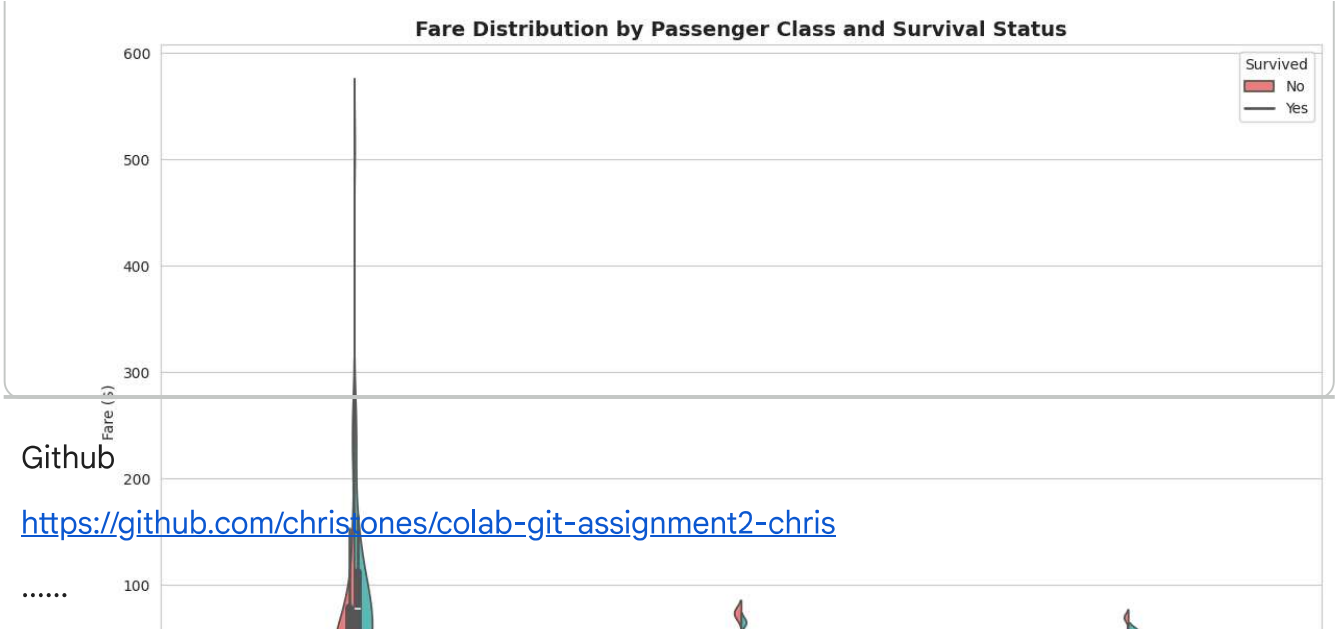
```
=====
TASK 9: Analyze age distribution
=====
```



Age statistics by passenger class and sex:

Pclass	Sex	count	mean	std	min	25%	50%	75%	max
1	female	94.0	35.255319	13.257529	2.00	24.00	35.0	44.75	63.0
1	male	122.0	41.511639	14.324665	0.92	31.00	41.5	50.00	80.0
2	female	76.0	28.980263	12.855723	2.00	22.75	28.5	36.00	57.0
2	male	108.0	30.493796	14.382035	0.67	23.00	30.0	37.00	70.0
3	female	144.0	22.135417	12.181031	0.75	15.00	22.0	29.00	63.0
3	male	343.0	26.470612	11.843691	0.42	20.00	25.0	32.00	74.0

```
=====
TASK 10: Analyze fare distribution
=====
```



Github

<https://github.com/chrisones/colab-git-assignment2-chris>

.....